

6.1 Querying Databases the non-relational ways

The rise of non-relational DBMS

Handling Data Everywhere → NoSQL

- New kinds of data: web data, social networks, scientific data.
- New requirements:
 - Volume → scalability
 - Handling extremely large data.
 - Handling extremely many users.
- Variety → one model may not fit all.
 - Handling very simple to very complex data.
- NoSQL databases
 - originally "non SQL" or "non relational"
 - Now "not only SQL"

How to query Non-relations?

Querying Relations: Key Concepts

Concept	operated Upon	Example
Reducing	... tuples and attributes	SELECT beer WHERE brewer = "AB In Bev"
Combining	... two relations	FROM Beers, Sells
Grouping	... tuples by attributes	GROUP BY beer
Aggregating	... attributes across tuples	COUNT(bar)
Transforming	... attributes for result tuples	AVG(price) as AveragePrice

6.2 Querying document databases -

MongoDB

From Relations to Documents

SQL Term / Concept	MongoDB Term / Concept
database	database
table	collection
row	document or ^{BSON doc} row
column	field
index	index
table joins	\$ lookup, embedded documents
primary key	primary key

Querying MongoDB: Mechanism

- Queries are issued using "shell commands"
- You can access databases using interactive "Mongo Shell".
- Applications can access databases using "shell methods"
 - via client language drivers (python)

shell commands

```
show databases
show collections
use myDB
db.Bars.find() # shows all documents
```

MongoDB Compass ⇒ GUI

Querying MongoDB: Shell Methods

Method	Description
<code>db.collection.aggregate()</code>	Provides access to aggregation pipelines.
<code>db.collection.count()</code>	Return a count of the number of documents in a collection or view.
<code>db.collection.distinct()</code>	Returns an array of documents that have distinct values for the specified field.
<code>db.collection.find()</code>	Performs a query on a collection or a view and returns a cursor object.
<code>db.collection.findOne()</code>	Perform a query and returns a single document.
<code>db.collection.mapReduce()</code>	Performs MapReduce style data aggregation.

Doc Querying: Collection Perspective

- All query methods are collection methods.
- What this means? A query is performed in either form:
 - operating within the scope of one collection
 - operating from one primary collection to another (asymmetrical)

Doc Querying: "Relational/SQL Capabilities"

- Much influence from relational/SQL capabilities
- Evident from the design, documentation and explanation throughout.

6.3 Querying Document Databases (36)

Basic operations

The basic Method: `db.collection.find()`

- MongoDB supports querying through several shell methods.
- Not totally complementary -- quite overlapping.
- `db.collection.find()` is the basic.

Querying a Collection: Find()

`db.collection.find(query, projection)`

query: specifies selection filter using query operators.

projection: specifies the fields to return in the documents that match the query filter.

Returns a cursor to the documents.

Query is given in the form of a JSON document.

ex. `db.Beers.find({brewer: "AB InBev"}, {name: 1})`
`db.Sells.find({price: {"$lt": 5}})`

Binary Operations: Union, Difference, Cartesian Product (Join)

- No way to perform operations over collections.
- There are "similar operations" (e.g. `$setUnion`) for combining results within a collection.
- Binary Operations can be done with python, MongoDB does not support joins or binary operations with more than one collection. Documents are denormalized or stored with related data in documents.

Renaming

- Not supported in the basic find() command.
- However, supported in a similar construct in Aggregate.

Find() method compared.

- db.collection.find(selection, projection)

Relation Operation	Supported in db.collection.find()
Selection	Yes
Projection	Yes
Set Union	No
Set Difference	No
Cartesian Product	No
Renaming	No

6.4 Querying Document Databases - Handling Complex Structures

- We learned to "normalize" in relational structure.
- The document model assumes "denormalized" document structure.

Simple Document vs Complex Document
 ↓
 stores all related info within the document.

Querying Embedded Documents

- Condition = Attribute Operator Value
- Complex Values
 - Q: Find drinkers who frequent a bar with name X, address Y, and owner Z.
 - db.CompleteDrinkers.find({

frequent: {name: "Green Bar", addr: "Green st", owner: "sally"}}, {name: 1})
- Complex Attributes
 - Q: Find drinkers who frequent a bar at address Y.
 - db.CompleteDrinkers.find({ "frequent.addr": "Green st"}, {name: 1})
- Complex Operators
 - Q: Find drinkers who drink all these beers A, B, C.
 - db.CompleteDrinkers.find({ "drinks.beer.name": { \$all: ["Bud", "Sam Adams"] } }, {name: 1})

Choice of Embedding or References

To query a referenced document you need to do it yourself.

For example, using python.

6.5 Querying Document Databases -

Aggregates

Aggregation Framework: Pipeline of stages

db.collection.aggregate(pipeline, options)

↓ ↓

array document

Pipeline Aggregation Stages

operator	Relational/SQL operator
\$match	selection
\$project	projection
\$group	group-by
\$unwind	ungroup-by
\$lookup	outer join

Aggregate: Includes and Generalizes find()

- Simple two-stage pipeline:

```
db.collection.aggregate([
  { $match: { conditions } }, { $project: { attributes } } ])
```

- Similar to db.collection.find(conditions, attributes)

```
• db.beers.find({ brewer: "AB InBev" },
  { "_id": 0, name: 1 } ])
```

- But more general / powerful

- \$project supports renaming, transformation.

```
• db.beers.aggregate([
  { $match: { brewer: "AB InBev" } },
  { $project: { "_id": 0, beer: "$name" } } ])
```

name renamed as beer.

Group-by

(38)

```
db.drinks.aggregate([ { $group: { "_id": "beer" } } ])
```

↳ groupby beer and assign to "_id" attribute.

```
db.drinks.aggregate([ { $group: { "_id": "beer",
  aggregating: { $avg: "$rating" } } } ])
```

unwind

```
db.CompleteBars.aggregate([ { $unwind:
  "sells" } ])
```

↳ Flattens sells to show as ^{another} many documents for each item in sells.

```
db.CompleteBars.aggregate([ { $unwind:
  "sells" }, { $match: { "sells.brewer":
  "AB InBev" } } ])
```

```
db.CompleteBars.aggregate([ { $unwind: "$sells" },
  { $match: { "sells.brewer": "AB InBev" } },
  { $group: { "_id": "$sells.beer.name", countSales:
    { $sum: 1 }, avgPrice: { $avg: "$sells.price" } } },
  { $match: { countSales: { $gt: 1 } } } ])
```

collection to collection using operators

6.6 Querying Document Databases -

(39)

The case for joins

- MongoDB added joins

Outer Equi-Join:

collection-centric perspective

- For every document in the left collection:
 - Find every document with equi-matching in the right collection.

db.bars.aggregate([

{ \$lookup:

{ from: "sells", localField: "name", foreignField: "bar", as: "menu" }},

{ \$project: { "_id": 0, "menu._id": 0, "menu.bar": 0 } }]